

ICS1513 - Computer Networks Laboratory

Experiment 6: Implementation of Error Detection using Checksum and CRC

Nitin H - 3122247001042

August 17 2026

Aim

To develop a socket programming application in C that simulates reliable data transmission using two error-detection techniques – Checksum (1's complement addition with end-around carry) and Cyclic Redundancy Check (CRC) – and to verify, at the receiver, whether the transmitted data was corrupted in transit.

Question Summary

Develop a socket programming application using C that simulates reliable data transmission using the following:

- **Checksum:** Divide the input data into 8-bit words, perform 1's complement addition with end-around carry, and generate the checksum at the sender side. At the receiver side, verify the received data using the checksum and determine whether an error has occurred. Display the intermediate sum, carry, checksum, and final verification result.
- **Cyclic Redundancy Check (CRC):** The sender converts the given data into its binary equivalent and selects a suitable generator polynomial (e.g. $G(x) = x^4 + x + 1$, i.e. 10011). The sender computes the CRC and appends it to the data. The data with CRC is transmitted through TCP sockets. The receiver recalculates the CRC using the same generator polynomial and reports whether the received frame is error-free or contains errors. Bit errors are manually introduced before verification to demonstrate CRC error detection.

1 Checksum

Server (Receiver) Algorithm

1. Create a TCP socket using `socket(AF_INET, SOCK_STREAM, 0)` and set `SO_REUSEADDR` using `setsockopt()`.
2. Bind the socket to `INADDR_ANY` on port 8080 using `bind()`, and put it into listening mode using `listen()`.
3. Accept an incoming sender connection using `accept()`.
4. Read the binary data string sent by the sender using `read()`.
5. Read the 8-bit checksum string sent by the sender using `read()`.
6. Call `verify_checksum()`:
 - (a) Append the received checksum to the received binary data, padded to a multiple of 8 bits.

- (b) Split the combined bit-string into 8-bit words and convert each word to its decimal value using `strtol()` with base 2.
 - (c) Accumulate the sum of all words. Whenever the running sum exceeds 0xFF (255), perform an end-around carry: add the overflow bits back into the lower 8 bits of the sum.
 - (d) Print each word, its decimal value, and the intermediate sum at every step.
 - (e) If the final accumulated sum equals 0xFF (all 1s), the data is error-free; otherwise an error is detected.
7. Print the final result ("No error detected" or "ERROR DETECTED").
 8. Send an ACK ("ACK: No error detected") or NACK ("NACK: Error detected in data") back to the sender using `send()`.
 9. Close the client and server sockets.

Client (Sender) Algorithm

1. Prompt the user for the server's IP address and the text data to be sent.
2. Convert the input text to its binary equivalent (8 bits per character) using `text_to_binary()`.
3. Create a TCP socket, populate a `sockaddr_in` structure with the server's IP and port 8080, and `connect()` to the server.
4. Call `compute_checksum()`:
 - (a) Pad the binary data to a multiple of 8 bits, then split it into 8-bit words.
 - (b) Convert each word to its decimal value and accumulate the sum, applying end-around carry whenever the sum exceeds 0xFF, printing each intermediate step.
 - (c) Compute the final checksum as the 1's complement (bitwise NOT) of the final 8-bit sum.
5. Prompt the user to optionally inject a bit error into the binary data (flipping a bit at a user-specified position) to simulate transmission corruption.
6. Send the (possibly corrupted) binary data to the receiver using `send()`, followed by the computed 8-bit checksum using `send()`.
7. Read the receiver's ACK/NACK verdict using `read()` and print it to the console.
8. Close the socket.

Server Code (Q61Server.c)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/socket.h>
5  #include <netinet/in.h>
6  #include <arpa/inet.h>
7  #include <unistd.h>
8
9  #define PORT 8080
10 #define MAXLEN 4096
11
12 /*
13  * Adds all 8-bit words of data INCLUDING the received checksum word
14  * using 1's complement addition with end-around carry.
15  * If the final sum is all 1s (0xFF), the data is error-free.
16  */
17 static int verify_checksum(const char *binary_data, const char *received_checksum_bin) {

```

```

18 char padded[MAXLEN];
19 strcpy(padded, binary_data);
20 while ((int)strlen(padded) % 8 != 0) strcat(padded, "0");
21 strcat(padded, received_checksum_bin);
22
23 int nwords = strlen(padded) / 8;
24 unsigned int sum = 0;
25
26 printf("\n---ChecksumVerification(Receiver)---\n");
27 for (int i = 0; i < nwords; i++) {
28     char word[9];
29     strncpy(word, padded + i * 8, 8);
30     word[8] = '\0';
31     unsigned int val = (unsigned int)strtol(word, NULL, 2);
32     printf("Word%d: %s(decimal %u)\n", i + 1, word, val);
33
34     sum += val;
35     if (sum > 0xFF) {
36         unsigned int carry = sum >> 8;
37         sum = (sum & 0xFF) + carry;
38         printf("Carry generated -> end-around carry added. Intermediate sum = %u\n", sum);
39     } else {
40         printf("Intermediate sum = %u\n", sum);
41     }
42 }
43
44 printf("Final accumulated sum = %u (0x%02X)\n", sum, sum);
45 return (sum == 0xFF); /* all 1s -> no error */
46 }
47
48 int main() {
49     int server_fd, client_fd;
50     struct sockaddr_in addr;
51     socklen_t addrlen = sizeof(addr);
52
53     server_fd = socket(AF_INET, SOCK_STREAM, 0);
54     if (server_fd < 0) { perror("socket"); exit(1); }
55
56     int opt = 1;
57     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
58
59     addr.sin_family = AF_INET;
60     addr.sin_addr.s_addr = INADDR_ANY;
61     addr.sin_port = htons(PORT);
62
63     if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
64         perror("bind"); exit(1);
65     }
66     if (listen(server_fd, 1) < 0) { perror("listen"); exit(1); }
67
68     printf("ChecksumReceiver listening on port %d...\n", PORT);
69
70     client_fd = accept(server_fd, (struct sockaddr *)&addr, &addrlen);
71     if (client_fd < 0) { perror("accept"); exit(1); }
72     printf("Sender connected.\n");
73
74     /* 1. Receive binary data */
75     char binary_data[MAXLEN];
76     int n = read(client_fd, binary_data, sizeof(binary_data) - 1);
77     binary_data[n] = '\0';
78
79     /* 2. Receive checksum (8-bit binary string) */
80     char checksum_bin[16];
81     n = read(client_fd, checksum_bin, sizeof(checksum_bin) - 1);
82     checksum_bin[n] = '\0';
83
84     printf("\nReceived binary data: %s\n", binary_data);
85     printf("Received checksum: %s\n", checksum_bin);

```

```

86
87     int ok = verify_checksum(binary_data, checksum_bin);
88
89     printf("\n=====RESULT=====\n");
90     printf(ok ? "No_error_detected_Data_is_intact.\n"
91           : "ERROR_DETECTED!Data_is_corrupted.\n");
92     printf("=====\\n");
93
94     char *reply = ok ? "ACK:No_error_detected"
95                     : "NACK:Error_detected_in_data";
96     send(client_fd, reply, strlen(reply), 0);
97
98     close(client_fd);
99     close(server_fd);
100     return 0;
101 }

```

Client Code (Q61Client.c)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/socket.h>
5  #include <netinet/in.h>
6  #include <arpa/inet.h>
7  #include <unistd.h>
8
9  #define PORT 8080
10 #define MAXLEN 4096
11
12 /* Convert an ASCII string to a binary string (8 bits per char) */
13 static void text_to_binary(const char *text, char *binout) {
14     binout[0] = '\0';
15     for (size_t i = 0; i < strlen(text); i++) {
16         char byte[9];
17         unsigned char c = (unsigned char)text[i];
18         for (int b = 7; b >= 0; b--) {
19             byte[7 - b] = ((c >> b) & 1) ? '1' : '0';
20         }
21         byte[8] = '\0';
22         strcat(binout, byte);
23     }
24 }
25
26 /* Flips one bit in a binary string, for error-injection demo */
27 static void inject_error(char *binstr, int pos) {
28     if (pos < 0 || pos >= (int)strlen(binstr)) return;
29     binstr[pos] = (binstr[pos] == '0') ? '1' : '0';
30 }
31
32 /*
33  * Splits binary data into 8-bit words, adds them using 1's complement
34  * arithmetic with end-around carry, and produces the checksum
35  * (1's complement of the final sum) as an 8-bit binary string.
36  */
37 static void compute_checksum(const char *binary_data, char *checksum_out) {
38     int len = strlen(binary_data);
39     int nwords = (len + 7) / 8;
40
41     char padded[MAXLEN];
42     strcpy(padded, binary_data);
43     while ((int)strlen(padded) % 8 != 0) strcat(padded, "0");
44
45     printf("\n---Checksum Computation(Sender)---\\n");
46     printf("Total_8-bit_words:_%d\\n", nwords);
47

```

```

48 unsigned int sum = 0;
49 for (int i = 0; i < nwords; i++) {
50     char word[9];
51     strncpy(word, padded + i * 8, 8);
52     word[8] = '\0';
53     unsigned int val = (unsigned int)strtol(word, NULL, 2);
54     printf("Word%d: %s (decimal %u)\n", i + 1, word, val);
55
56     sum += val;
57     if (sum > 0xFF) {
58         unsigned int carry = sum >> 8;
59         sum = (sum & 0xFF) + carry;
60         printf("Carry generated -> added back (end-around carry). Intermediate sum = %u\n", sum);
61     } else {
62         printf("Intermediate sum = %u\n", sum);
63     }
64 }
65
66 unsigned int checksum = (~sum) & 0xFF;
67
68 char sum_bin[9], chk_bin[9];
69 for (int b = 7; b >= 0; b--) sum_bin[7 - b] = ((sum >> b) & 1) ? '1' : '0';
70 sum_bin[8] = '\0';
71 for (int b = 7; b >= 0; b--) chk_bin[7 - b] = ((checksum >> b) & 1) ? '1' : '0';
72 chk_bin[8] = '\0';
73
74 printf("Final Sum (bin) = %s (decimal %u)\n", sum_bin, sum);
75 printf("Checksum (1's complement of sum) = %s (decimal %u)\n", chk_bin, checksum);
76
77 strcpy(checksum_out, chk_bin);
78 }
79
80 int main() {
81     int sock;
82     struct sockaddr_in serv_addr;
83     char server_ip[64];
84     char input[MAXLEN];
85     char binary_data[MAXLEN];
86
87     printf("Enter server IP (e.g. 127.0.0.1): ");
88     scanf("%63s", server_ip);
89     getchar();
90
91     printf("Enter data to send (text): ");
92     fgets(input, sizeof(input), stdin);
93     input[strcspn(input, "\n")] = '\0';
94
95     text_to_binary(input, binary_data);
96     printf("\nText: %s\n", input);
97     printf("Binary: %s\n", binary_data);
98
99     /* --- connect --- */
100     sock = socket(AF_INET, SOCK_STREAM, 0);
101     if (sock < 0) { perror("socket"); exit(1); }
102
103     serv_addr.sin_family = AF_INET;
104     serv_addr.sin_port = htons(PORT);
105     if (inet_pton(AF_INET, server_ip, &serv_addr.sin_addr) <= 0) {
106         perror("invalid address"); exit(1);
107     }
108     if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
109         perror("connect"); exit(1);
110     }
111     printf("Connected to receiver.\n");
112
113     char checksum_bin[16];
114     compute_checksum(binary_data, checksum_bin);
115

```

```

116     printf("\nInject a bit error before sending? (1=Yes, 0=No): ");
117     int corrupt; scanf("%d", &corrupt);
118     char to_send[MAXLEN];
119     strcpy(to_send, binary_data);
120     if (corrupt) {
121         int pos;
122         printf("Enter bit position to flip (0-%d): ", (int)strlen(to_send) - 1);
123         scanf("%d", &pos);
124         inject_error(to_send, pos);
125         printf("Corrupted data: %s\n", to_send);
126     }
127
128     send(sock, to_send, strlen(to_send), 0);
129     send(sock, checksum_bin, strlen(checksum_bin), 0);
130
131     /* --- receive verdict from receiver --- */
132     char reply[256];
133     int n = read(sock, reply, sizeof(reply) - 1);
134     reply[n] = '\0';
135     printf("\nReceiver says: %s\n", reply);
136
137     close(sock);
138     return 0;
139 }

```

Sample Input and Output – No Error Injected

Input: Server IP 127.0.0.1, data "Nitin", inject error = 0

Output:

Text : Nitin

Binary: 0100111001101001011101000110100101101110

--- Checksum Computation (Sender) ---

Total 8-bit words: 5

...

Final Sum (bin) = 00000100 (decimal 4)

Checksum (1's complement of sum) = 11111011 (decimal 251)

Receiver says: ACK: No error detected

```

(Nightingales@LAPTOP-Q6HDMKCT) - [ /mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6 ]
$ ./a.out
Checksum Receiver listening on port 8080...
Sender connected.

Received binary data : 0100111001101001011101000110100101101110
Received checksum : 11111011

--- Checksum Verification (Receiver) ---
Word 1: 01001110 (decimal 78)
Intermediate sum = 78
Word 2: 01101001 (decimal 105)
Intermediate sum = 183
Word 3: 01110100 (decimal 116)
Carry generated -> end-around carry added. Intermediate sum = 44
Word 4: 01101001 (decimal 105)
Intermediate sum = 149
Word 5: 01101110 (decimal 110)
Carry generated -> end-around carry added. Intermediate sum = 4
Word 6: 11111011 (decimal 251)
Intermediate sum = 255
Final accumulated sum = 255 (0xFF)

===== RESULT =====
No error detected. Data is intact.
=====

(Nightingales@LAPTOP-Q6HDMKCT) - [ /mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6 ]
$

$ ./c
Enter server IP (e.g. 127.0.0.1): 127.0.0.1
Enter data to send (text): Nitin

Text : Nitin
Binary: 0100111001101001011101000110100101101110
Connected to receiver.

--- Checksum Computation (Sender) ---
Total 8-bit words: 5
Word 1: 01001110 (decimal 78)
Intermediate sum = 78
Word 2: 01101001 (decimal 105)
Intermediate sum = 183
Word 3: 01110100 (decimal 116)
Carry generated -> added back (end-around carry). Intermediate sum = 44
Word 4: 01101001 (decimal 105)
Intermediate sum = 149
Word 5: 01101110 (decimal 110)
Carry generated -> added back (end-around carry). Intermediate sum = 4
Final Sum (bin) = 00000100 (decimal 4)
Checksum (1's complement of sum) = 11111011 (decimal 251)

Inject a bit error before sending? (1=Yes, 0=No): 0

Receiver says: ACK: No error detected

(Nightingales@LAPTOP-Q6HDMKCT) - [ /mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6 ]
$

```

Figure 1: Checksum receiver and sender output – no bit error injected, data verified as intact

Sample Input and Output – Error Injected

Input: Data "Nitin", inject error = 1, bit position = 21

Output:

Inject a bit error before sending? (1=Yes, 0=No): 1

Enter bit position to flip (0-39): 21

Corrupted data: 0100111001101001011100001101001011101100

Receiver says: NACK: Error detected in data

```
(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6]
$ ./a.out
Checksum Receiver listening on port 8080...
Sender connected.

Received binary data : 0100111001101001011100001101001011110
Received checksum    : 11111011

--- Checksum Verification (Receiver) ---
Word 1: 01001110 (decimal 78)
      Intermediate sum = 78
Word 2: 01101001 (decimal 105)
      Intermediate sum = 183
Word 3: 01110000 (decimal 112)
      Carry generated -> end-around carry added. Intermediate sum = 40
Word 4: 01101001 (decimal 105)
      Intermediate sum = 145
Word 5: 01101110 (decimal 110)
      Intermediate sum = 255
Word 6: 11111011 (decimal 251)
      Carry generated -> end-around carry added. Intermediate sum = 251
Final accumulated sum = 251 (0xFB)

===== RESULT =====
ERROR DETECTED! Data is corrupted.
=====

(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6]
$

Enter data to send (text): Nitin

Text : Nitin
Binary: 0100111001101001011101000110100101101110
Connected to receiver.

--- Checksum Computation (Sender) ---
Total 8-bit words: 5
Word 1: 01001110 (decimal 78)
      Intermediate sum = 78
Word 2: 01101001 (decimal 105)
      Intermediate sum = 183
Word 3: 01110100 (decimal 116)
      Carry generated -> added back (end-around carry). Intermediate sum = 44
Word 4: 01101001 (decimal 105)
      Intermediate sum = 149
Word 5: 01101110 (decimal 110)
      Carry generated -> added back (end-around carry). Intermediate sum = 4
Final Sum (bin) = 00000100 (decimal 4)
Checksum (1's complement of sum) = 11111011 (decimal 251)

Inject a bit error before sending? (1=Yes, 0=No): 1
Enter bit position to flip (0-39): 21
Corrupted data: 0100111001101001011100001101001011101110

Receiver says: NACK: Error detected in data

(Nightingales@LAPTOP-Q6HDMKCT) - [/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6]
$
```

Figure 2: Checksum receiver and sender output – bit error injected at position 21, error correctly detected

2 Cyclic Redundancy Check (CRC)

Server (Receiver) Algorithm

1. Create a TCP socket, set `SO_REUSEADDR`, bind to port 8080, and `listen()` for a connection.
2. Accept an incoming sender connection using `accept()`.
3. Read the generator polynomial string sent by the sender using `read()`.
4. Read the transmitted frame (data bits followed by the CRC remainder bits) using `read()`.
5. Call `verify_crc()`:
 - (a) Perform modulo-2 (XOR-based) binary division of the received frame by the generator polynomial using `mod2_divide()`.
 - (b) Examine the remainder of the division.
 - (c) If the remainder is all zero bits, the frame is declared error-free; otherwise it is declared to contain errors.
6. Print the received frame, the remainder, and the final verification result.
7. Send an ACK ("ACK: Frame error-free") or NACK ("NACK: Frame contains errors") back to the sender using `send()`.
8. Close the client and server sockets.

Client (Sender) Algorithm

1. Prompt the user for the server's IP address and the text data to be sent.
2. Convert the input text to its binary equivalent using `text_to_binary()`.
3. Create a TCP socket and `connect()` to the server; send the generator polynomial string ("10011", corresponding to $G(x) = x^4 + x + 1$) using `send()`.
4. Call `compute_crc()`:
 - (a) Append $(n - 1)$ zero bits to the binary data, where n is the length of the generator polynomial.
 - (b) Perform modulo-2 division of the zero-appended data by the generator polynomial using `mod2_divide()`, obtained via repeated XOR operations (`xor_bits()`) wherever the leading bit of the current remainder is 1.
 - (c) The remainder of this division is the computed CRC checksum.
5. Append the computed CRC to the original binary data to form the transmitted frame.
6. Prompt the user to optionally inject a bit error into the frame (flipping a bit at a user-specified position) to simulate transmission corruption.
7. Send the (possibly corrupted) frame to the receiver using `send()`.
8. Read the receiver's ACK/NACK verdict using `read()` and print it to the console.
9. Close the socket.

Server Code (Q62Server.c)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/socket.h>
5  #include <netinet/in.h>
6  #include <arpa/inet.h>
7  #include <unistd.h>
8
9  #define PORT 8080
10 #define MAXLEN 4096
11
12 /* XOR two binary strings of equal length */
13 static void xor_bits(const char *a, const char *b, char *result, int len) {
14     for (int i = 0; i < len; i++)
15         result[i] = (a[i] == b[i]) ? '0' : '1';
16     result[len] = '\0';
17 }
18
19 /* Modulo-2 division of dividend by generator; returns remainder */
20 static void mod2_divide(const char *dividend, const char *generator, char *remainder_out) {
21     int dlen = strlen(dividend);
22     int glen = strlen(generator);
23
24     char *temp = (char *)malloc(dlen + 1);
25     strcpy(temp, dividend);
26
27     for (int i = 0; i <= dlen - glen; ) {
28         if (temp[i] == '1') {
29             char part[MAXLEN], xored[MAXLEN];
30             strncpy(part, temp + i, glen);
31             part[glen] = '\0';
32             xor_bits(part, generator, xored, glen);
33             memcpy(temp + i, xored, glen);
34         }
35         i++;
36     }
37
38     strncpy(remainder_out, temp + (dlen - (glen - 1)), glen - 1);
39     remainder_out[glen - 1] = '\0';
40     free(temp);
41 }
42
43 /* Divides received frame (data+CRC) by generator; remainder all-zero => no error */
44 static int verify_crc(const char *received_frame, const char *generator) {
45     char remainder[MAXLEN];
46     printf("\n---CRC Verification (Receiver)---\n");
47     printf("Received frame (data+CRC): %s\n", received_frame);
48
49     mod2_divide(received_frame, generator, remainder);
50     printf("Remainder after division: %s\n", remainder);
51
52     for (size_t i = 0; i < strlen(remainder); i++)
53         if (remainder[i] != '0') return 0;
54     return 1;
55 }
56
57 int main() {
58     int server_fd, client_fd;
59     struct sockaddr_in addr;
60     socklen_t addrlen = sizeof(addr);
61
62     server_fd = socket(AF_INET, SOCK_STREAM, 0);
63     if (server_fd < 0) { perror("socket"); exit(1); }
64
65     int opt = 1;
66     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

```

67     addr.sin_family = AF_INET;
68     addr.sin_addr.s_addr = INADDR_ANY;
69     addr.sin_port = htons(PORT);
70
71
72     if (bind(server_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
73         perror("bind"); exit(1);
74     }
75     if (listen(server_fd, 1) < 0) { perror("listen"); exit(1); }
76
77     printf("CRCReceiver_listening_on_port%d...\n", PORT);
78
79     client_fd = accept(server_fd, (struct sockaddr *)&addr, &addrlen);
80     if (client_fd < 0) { perror("accept"); exit(1); }
81     printf("Sender_connected.\n");
82
83     /* 1. Receive generator */
84     char generator[32];
85     int n = read(client_fd, generator, sizeof(generator) - 1);
86     generator[n] = '\0';
87
88     /* 2. Receive frame (data + CRC bits), possibly corrupted */
89     char frame[MAXLEN];
90     n = read(client_fd, frame, sizeof(frame) - 1);
91     frame[n] = '\0';
92
93     printf("\nGenerator_received: %s\n", generator);
94     printf("Frame_received: %s\n", frame);
95
96     int ok = verify_crc(frame, generator);
97
98     printf("\n=====RESULT=====\\n");
99     printf(ok ? "Frame_is_ERROR-FREE.\\n"
100           : "Frame_CONTAINS_ERROR(S)!\\n");
101     printf("=====\\n");
102
103     char *reply = ok ? "ACK: Frame_error-free"
104                     : "NACK: Frame_contains_errors";
105     send(client_fd, reply, strlen(reply), 0);
106
107     close(client_fd);
108     close(server_fd);
109     return 0;
110 }

```

Client Code (Q62Client.c)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/socket.h>
5  #include <netinet/in.h>
6  #include <arpa/inet.h>
7  #include <unistd.h>
8
9  #define PORT 8080
10 #define GENERATOR "10011" /* G(x) = x^4 + x + 1 */
11 #define MAXLEN 4096
12
13 /* Convert an ASCII string to a binary string (8 bits per char) */
14 static void text_to_binary(const char *text, char *binout) {
15     binout[0] = '\0';
16     for (size_t i = 0; i < strlen(text); i++) {
17         char byte[9];
18         unsigned char c = (unsigned char)text[i];
19         for (int b = 7; b >= 0; b--) {

```

```

20     byte[7 - b] = ((c >> b) & 1) ? '1' : '0';
21 }
22     byte[8] = '\0';
23     strcat(binout, byte);
24 }
25 }
26
27 /* Flips one bit in a binary string, for error-injection demo */
28 static void inject_error(char *binstr, int pos) {
29     if (pos < 0 || pos >= (int)strlen(binstr)) return;
30     binstr[pos] = (binstr[pos] == '0') ? '1' : '0';
31 }
32
33 /* XOR two binary strings of equal length */
34 static void xor_bits(const char *a, const char *b, char *result, int len) {
35     for (int i = 0; i < len; i++)
36         result[i] = (a[i] == b[i]) ? '0' : '1';
37     result[len] = '\0';
38 }
39
40 /* Modulo-2 division of dividend by generator; returns remainder */
41 static void mod2_divide(const char *dividend, const char *generator, char *remainder_out) {
42     int dlen = strlen(dividend);
43     int glen = strlen(generator);
44
45     char *temp = (char *)malloc(dlen + 1);
46     strcpy(temp, dividend);
47
48     for (int i = 0; i <= dlen - glen; ) {
49         if (temp[i] == '1') {
50             char part[MAXLEN], xored[MAXLEN];
51             strncpy(part, temp + i, glen);
52             part[glen] = '\0';
53             xor_bits(part, generator, xored, glen);
54             memcpy(temp + i, xored, glen);
55         }
56         i++;
57     }
58
59     strncpy(remainder_out, temp + (dlen - (glen - 1)), glen - 1);
60     remainder_out[glen - 1] = '\0';
61     free(temp);
62 }
63
64 /* Computes CRC for given binary data using the generator; prints steps */
65 static void compute_crc(const char *binary_data, const char *generator, char *crc_out) {
66     int glen = strlen(generator);
67     char appended[MAXLEN];
68     strcpy(appended, binary_data);
69     for (int i = 0; i < glen - 1; i++) strcat(appended, "0");
70
71     printf("\n---CRC Computation (Sender)---\n");
72     printf("Data: %s\n", binary_data);
73     printf("Generator: %s\n", generator);
74     printf("Data + %d zero bits: %s\n", glen - 1, appended);
75
76     mod2_divide(appended, generator, crc_out);
77     printf("Computed CRC (remainder) = %s\n", crc_out);
78 }
79
80 int main() {
81     int sock;
82     struct sockaddr_in serv_addr;
83     char server_ip[64];
84     char input[MAXLEN];
85     char binary_data[MAXLEN];
86
87     printf("Enter server IP (e.g. 127.0.0.1): ");

```

```

88     scanf("%63s", server_ip);
89     getchar();
90
91     printf("Enter_data_to_send(text):\n");
92     fgets(input, sizeof(input), stdin);
93     input[strcspn(input, "\n")] = '\0';
94
95     text_to_binary(input, binary_data);
96     printf("\nText:\n%s\n", input);
97     printf("Binary:\n%s\n", binary_data);
98
99     /* --- connect --- */
100    sock = socket(AF_INET, SOCK_STREAM, 0);
101    if (sock < 0) { perror("socket"); exit(1); }
102
103    serv_addr.sin_family = AF_INET;
104    serv_addr.sin_port = htons(PORT);
105    if (inet_pton(AF_INET, server_ip, &serv_addr.sin_addr) <= 0) {
106        perror("invalid address"); exit(1);
107    }
108    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
109        perror("connect"); exit(1);
110    }
111    printf("Connected to receiver.\n");
112
113    send(sock, GENERATOR, strlen(GENERATOR), 0);
114
115    char crc[32];
116    compute_crc(binary_data, GENERATOR, crc);
117
118    char frame[MAXLEN];
119    snprintf(frame, sizeof(frame), "%s%s", binary_data, crc);
120    printf("Transmitted frame (data+CRC):\n%s\n", frame);
121
122    printf("\nInject a bit error before sending? (1=Yes, 0=No):\n");
123    int corrupt; scanf("%d", &corrupt);
124    if (corrupt) {
125        int pos;
126        printf("Enter bit position to flip (0-%d):\n", (int)strlen(frame) - 1);
127        scanf("%d", &pos);
128        inject_error(frame, pos);
129        printf("Corrupted frame:\n%s\n", frame);
130    }
131
132    send(sock, frame, strlen(frame), 0);
133
134    /* --- receive verdict from receiver --- */
135    char reply[256];
136    int n = read(sock, reply, sizeof(reply) - 1);
137    reply[n] = '\0';
138    printf("\nReceiver says:\n%s\n", reply);
139
140    close(sock);
141    return 0;
142 }

```

Sample Input and Output – No Error Injected

Input: Data "Nitin", generator 10011, inject error = 0

Output:

```

Data           : 0100111001101001011101000110100101101110
Generator      : 10011
Computed CRC (remainder) = 1001
Transmitted frame (data+CRC) = 01001110011010010111010001101001011011101001

```

Receiver says: ACK: Frame error-free

```
(Nightingales@LAPTOP-Q6HDMKCT)-[/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6]
$ ./s
CRC Receiver listening on port 8080...
Sender connected.

Generator received : 10011
Frame received      : 01001110011010010111010001101001011011101001

--- CRC Verification (Receiver) ---
Received frame (data+CRC): 01001110011010010111010001101001011011101001
Remainder after division : 0000

===== RESULT =====
Frame is ERROR-FREE.
=====

(Nightingales@LAPTOP-Q6HDMKCT)-[/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6]
$

Enter data to send (text): Nitin

Text : Nitin
Binary: 0100111001101001011101000110100101101110
Connected to receiver.

--- CRC Computation (Sender) ---
Data      : 0100111001101001011101000110100101101110
Generator : 10011
Data + 4 zero bits: 01001110011010010111010001101001011011100000
Computed CRC (remainder) = 1001
Transmitted frame (data+CRC) = 01001110011010010111010001101001011011101001

Inject a bit error before sending? (1=Yes, 0=No): 0

Receiver says: ACK: Frame error-free

(Nightingales@LAPTOP-Q6HDMKCT)-[/mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6]
$
```

Figure 3: CRC receiver and sender output – no bit error injected, frame verified as error-free (remainder = 0000)

Sample Input and Output – Error Injected

Input: Data "Nitin", generator 10011, inject error = 1, bit position = 21

Output:

Inject a bit error before sending? (1=Yes, 0=No): 1

Enter bit position to flip (0-43): 21

Corrupted frame: 01001110011010010111000001101001011011101001

Receiver says: NACK: Frame contains errors

```
(Nightingales@LAPTOP-Q6HDMKCT)-[ /mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6 ]
$ ./s
CRC Receiver listening on port 8080...
Sender connected.

Generator received : 10011
Frame received    : 01001110011010010111000001101001011011101001

--- CRC Verification (Receiver) ---
Received frame (data+CRC): 01001110011010010111000001101001011011101001
Remainder after division : 1011

===== RESULT =====
Frame CONTAINS ERROR(S)!
=====

(Nightingales@LAPTOP-Q6HDMKCT)-[ /mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6 ]
$ 
$ ./a.out
Enter server IP (e.g. 127.0.0.1): 127.0.0.1
Enter data to send (text): Nitin

Text : Nitin
Binary: 0100111001101001011101000110100101101110
Connected to receiver.

--- CRC Computation (Sender) ---
Data      : 0100111001101001011101000110100101101110
Generator : 10011
Data + 4 zero bits: 01001110011010010111010001101001011011100000
Computed CRC (remainder) = 1001
Transmitted frame (data+CRC) = 01001110011010010111010001101001011011101001

Inject a bit error before sending? (1=Yes, 0=No): 1
Enter bit position to flip (0-43): 21
Corrupted frame: 01001110011010010111000001101001011011101001

Receiver says: NACK: Frame contains errors

(Nightingales@LAPTOP-Q6HDMKCT)-[ /mnt/c/Users/Nitin/Desktop/Semester 5/ICS1513-Computer-Networks-Labratory/Experiment 6 ]
$
```

Figure 4: CRC receiver and sender output – bit error injected at position 21, non-zero remainder (1011) correctly flags the error

Learning Outcomes

- Learnt to implement checksum-based error detection using 1's complement addition with end-around carry, at both the sender and receiver ends, over a TCP socket connection.
- Learnt to implement CRC-based error detection using modulo-2 (XOR) binary division against a chosen generator polynomial, at both the sender and receiver ends, over a TCP socket connection.

- Understood how deliberately injecting bit errors before transmission demonstrates the error-detecting capability (and limitations) of both the checksum and CRC techniques.
- Learnt to interpret the intermediate computation steps – word-wise sums, end-around carries, and division remainders – that lead to the final checksum/CRC value and verification result.